An Explainable Machine Learning Framework for Phishing Website Detection Using URL and Network Features

Tanay Samson Pathare[1]

[1](*Department of Computer Science, Bhonsala Military College, Nashik, Maharashtra, India)*

Corresponding Author: tanaypathare1@gmail.com

Abstract: Phishing websites represent a pervasive cyber threat engineered to harvest credentials through deceptive web infrastructure. While machine learning enables proactive detection beyond signature blacklists, operational deployment is hindered by model opacity and the latency of external domain inspection. This paper presents an explainable machine learning framework for phishing website detection using 111 structured URL lexical, path, query, and network resolver features on a benchmark corpus of 58,645 URLs. Evaluating five classification architectures across 11,729 held-out test instances with 5-fold cross-validation on the training set, XGBoost achieves superior discrimination, yielding 94.82% accuracy and 95.05% F1-score (5-seed mean: 94.74% ± 0.08% accuracy, 94.98% ± 0.08% F1-score), statistically outperforming Random Forest ($p < 0.001$, McNemar's test). Tree-based Shapley Additive Explanations (TreeSHAP) applied strictly to training data ensure train-only attribution, revealing high ranking stability under retraining and identifying directory length, domain registration age, and delimiters as primary discriminators. Feature reduction demonstrates that a compact top-20 subset preserves full test performance while lowering model inference latency to 1.67 ms per 1,000 queries. Furthermore, an uncertainty-gated cascade resolves 64.3% of URLs in-memory with 97.78% accuracy using zero-network lexical parsing, establishing an operationally viable tiered triage architecture that eliminates nearly two-thirds of external resolver lookups.

Key Word: Phishing Website Detection; Explainable Artificial Intelligence (XAI); Machine Learning; URL Lexical Analysis; TreeSHAP; Feature Selection; Cybersecurity.

I. Introduction

Phishing represents a pervasive threat in modern cyberspace. Attackers engineer deceptive URLs and spoofed web portals to impersonate authentic organizations, harvesting credentials and financial assets [1], [2]. Threat telemetry from the Anti-Phishing Working Group (APWG) [1] documents millions of active phishing attacks annually, while the Verizon Data Breach Investigations Report [2] identifies credential theft as a primary breach vector across confirmed corporate breaches.

Conventional defenses rely heavily on centralized signature databases and community blacklists (e.g., Google Safe Browsing, PhishTank). While blacklists provide high precision on cataloged domains, they exhibit significant discovery delays, often requiring hours to verify zero-hour domains [3]-[6], during which zero-day attacks achieve maximum impact.

To overcome blacklist latency, machine learning (ML) has emerged as a proactive alternative, classifying domain, path, and structural indicators dynamically [7], [8]. However, operational adoption in Security Operations Centers (SOCs) faces two critical bottlenecks: (i) Model Opacity, where opaque tree ensembles fail to provide interpretable feature attributions for containment decisions; and (ii) Page Crawling Latency, where full DOM rendering introduces substantial latency and risks drive-by malware [9], [10].

To address these challenges, this paper presents an empirical benchmark and explainable ML framework for phishing detection using 111 structured URL lexical, path, file, query, and resolver metrics on 58,645 URLs [11]. We evaluate five paradigms: Logistic Regression, Decision Trees, Random Forests, Linear SVM, and XGBoost [12] across 11,729 held-out test samples with 5-fold cross-validation. To ensure transparency, we apply Tree-based Shapley Additive Explanations (TreeSHAP) [13], [14] strictly on training data to ensure train-only attribution.

The primary contributions of this work are:

1. Multi-Model Benchmark & Statistical Validation: We evaluate five supervised ML architectures across 11,729 test instances with 5-fold CV on training data (N=46,916), verifying that XGBoost's lead (5-seed mean: 94.74% ± 0.08% acc, 94.98% ± 0.08% F1; seed 42: 94.82% accuracy, 95.05% F1) over Random Forest is statistically significant via McNemar's test ($p < 0.001$).
2. Train-Only Attribution & Ranking Stability: We implement TreeSHAP strictly on training data to ensure train-only attribution, demonstrating robust ranking stability under model retraining ($\rho_s = 0.973 \pm 0.010$ across 98 features, Kendall's $\tau = 0.795$ within top-20, 95.0% top-20 set overlap; $\rho_s = 0.9991$ under explanation subsampling) and identifying primary signals (directory length, domain activation age, delimiters).
3. Feature Reduction & Cascaded Triage: We evaluate nested subsets, selecting k=20 via training CV as the smallest tested subset within 0.2 percentage points of full performance which sustains full predictive performance (5-seed test mean: 94.71% ± 0.09% acc, 94.95% ± 0.08% F1) while lowering model-only latency to 1.67 ms per 1,000 queries. Furthermore, an uncertainty-gated cascade demonstrates that a zero-network lexical-only model (84 features) resolves 64.3% of URLs in-memory with 97.78% accuracy, preserving 94.57% overall pipeline accuracy while cutting external network queries by 64.3%.
4. Computational Profiling: We measure model inference latency (mean ± std over 20 repeated runs), single-query latency (N=1), serialized model footprints, and throughput under batched execution.

II. Related Work

The evolution of automated phishing detection spans four major paradigms: heuristic and blacklist methods, content-based deep learning, URL/network-centric machine learning, and Explainable AI (XAI) frameworks.

Heuristic and Blacklist-Based Detection

Early defenses relied on signature repositories and community blacklists [3]. While services like PhishTank and Google Safe Browsing provide authoritative verdicts for cataloged threats, studies by Hannousse and Yahiouche [5] and Marchal et al. [6] demonstrate that blacklists suffer from significant discovery latency. Attackers routinely evade static blacklists via domain generation algorithms (DGAs), fast-flux DNS, and disposable top-level domains (TLDs) [3], allowing zero-hour phishing campaigns to harvest credentials before signatures propagate.

Content-Based and Deep Learning Approaches

To circumvent blacklist propagation delays, several studies proposed inspecting web page contents, DOM trees, and client-side scripts. Opara et al. [9] developed HTMLPhish, employing CNN-LSTM networks to analyze raw HTML document flows. Mohammad et al. [15] investigated self-structuring neural networks on page-level features. Adebowale et al. [10] integrated convolutional and recurrent architectures on website features, while Al-Alyan and Al-Ahmadi [16] evaluated a URL-only convolutional neural network trained on over two million URLs. Despite high reported accuracy, content-based techniques exhibit severe operational liabilities: (i) they mandate live connections to hostile servers, introducing drive-by malware exposure; (ii) adversary cloaking techniques serve benign decoys to automated crawlers; and (iii) fetching and rendering pages incurs heavy network and computational overhead, precluding wire-speed perimeter inspection.

URL and Network-Centric Machine Learning

To achieve high-throughput triage without page rendering, researchers transitioned toward feature engineering on URLs and network resolver metadata. Sahoo et al. [3] and Rao and Pais [4] established comprehensive taxonomies of URL lexical properties. Sahingoz et al. [7] evaluated tree classifiers and linear SVMs on lexical representations. Babagoli et al. [8] proposed heuristic non-linear regression strategies for phishing detection, while Chiew et al. [17] analyzed hybrid ensemble feature selection. Vrbančič et al. [11] standardized a benchmark containing 111 structured features across 58,645 URLs, capturing URL, domain, directory, file, query, and resolver metadata without requiring web page rendering.

Explainable Artificial Intelligence (XAI) in Cybersecurity

Recent works have actively explored post-hoc interpretability in phishing detection. Al-Fayoumi et al. [18] proposed XAI-PhD, leveraging TreeSHAP with ensemble models to interpret URL classification decisions and prune features to reduce prediction overhead. Uddin et al. [19] combined gradient-boosting models with SHAP on a public Kaggle dataset. Calzarossa et al. [20] proposed an explainable machine learning procedure to identify critical phishing indicators, while Kehkashan et al. [21] evaluated random forest with SHAP on a Kaggle dataset. While these studies established the utility of post-hoc explanations, several methodological distinctions and gaps remain: (i) prior works in this domain, including XAI-PhD [18], do not explicitly specify whether global SHAP attributions and subsequent feature pruning are restricted strictly to an independent training partition or evaluated globally across the dataset prior to final validation, risking subtle selection leakage if test instances inform feature pruning; (ii) attribution ranking stability across random seeds and subsamples is often unquantified; (iii) existing feature selection methods treat all features uniformly, whereas synchronous external network lookups (DNS/WHOIS) incur tens to hundreds of milliseconds of latency compared to microsecond in-memory lexical parsing; and (iv) computational latency reporting frequently omits repeated measurement intervals or single-query benchmarking. This paper addresses these challenges by enforcing a strict train-only TreeSHAP protocol on the standardized 58,645 URL corpus [11], verifying attribution stability across seeds, and evaluating an uncertainty-gated cascade that explicitly decouples in-memory lexical parsing from network resolver queries.

III. Materials and Methods

Study Design and System Architecture

The architectural pipeline of the proposed framework (Fig. 1) operates across four modular phases: (i) static feature extraction and zero-variance filtering, (ii) multi-model supervised classification, (iii) train-only TreeSHAP feature attribution on training data, and (iv) SHAP-directed feature pruning and computational profiling.

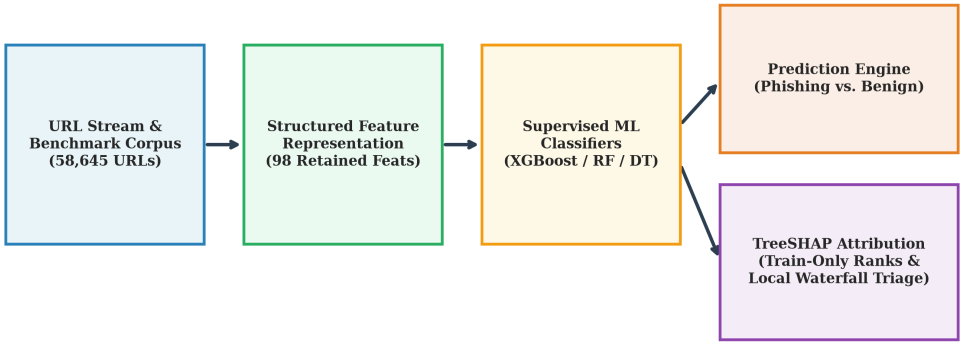


Fig. 1: Architectural Pipeline of the Proposed Explainable Phishing Detection Framework.

Dataset Specification and Preprocessing

We utilize the standardized benchmark corpus curated by Vrbančič et al. [11], comprising 58,645 URLs (30,647 confirmed phishing instances sourced from PhishTank and OpenPhish, and 27,998 benign instances from Alexa top global domains, search engine results, and open directories). The dataset establishes an 80% stratified training partition (46,916 URLs) and a 20% untouched held-out testing partition (11,729 URLs). Table no 1 outlines the complete dataset specification.

Table no 1: Shows Dataset Specification and Stratified Partitioning Parameters.

Property	Specification / Value
Dataset Provenance	Vrbančič et al. (Data in Brief, 2020) [11]
Feature Modality	Static URL Lexical, Path, File, Query, and Resolver
Total URL Instances	58,645 URLs
Raw Feature Dimensions	111 Numeric Features (97 Lexical / 14 Resolver; 96/15 in [11])
Retained Dimensions	98 Informative Features (13 Constant Delimiters Filtered)
Class Balance	30,647 Phishing (52.26%) / 27,998 Benign (47.74%)
Train / Test Partition	80% Stratified Train (46,916) / 20% Test (11,729)
Missing / Sentinel Values	0 NaN (Unset/unavailable metrics encoded as -1 sentinel)

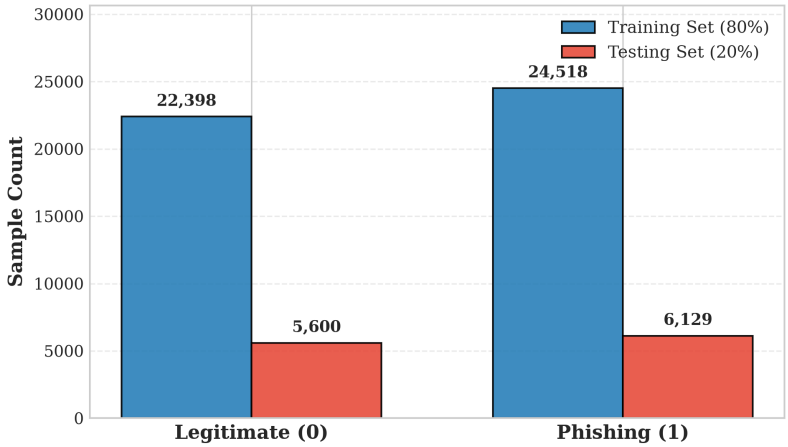


Fig. 2: Class Distribution Across Stratified Training and Testing Partitions.

A cross-partition feature vector audit indicates that exactly 335 test instances (2.86% of the 11,729 test set) share identical 98-feature vectors with instances in the training set. Within partitions, 906 rows in train (1.93%) and 130 rows in test (1.11%) represent internal duplicates due to discrete structural metrics on pathless root domains with default values. Crucially, 332 of the 335 cross-partition twin pairs (99.1%) share identical ground-truth labels (with only 3 label conflicts). Evaluating on the strictly de-duplicated test subset (N = 11,394), XGBoost achieves 94.71% accuracy and 95.05% F1-score, while Random Forest achieves 94.18% accuracy and 94.58% F1-score (compared to 94.82% / 95.05% and 94.30% / 94.57% on the full test set), confirming that cross-partition repetition does not inflate generalization metrics. The numeric sentinel -1 denotes 'not applicable' or 'lookup unavailable' (e.g., URLs lacking directory paths assign -1 to path metrics, and failed WHOIS lookups assign -1 to activation age). Preprocessing fits a zero-variance filter $V(X_j) > 0$ strictly on D_{train}. Thirteen constant features (all belonging to the domain structural family) are eliminated, leaving $D' = 98$ informative features (84 lexical/structural and 14 network/resolver metrics). For distance-based models (Logistic Regression and Linear SVM), features are standardized via z-score scaling fitted exclusively on D_{train}.

$$V(X_j) = (1 / N_{train}) \Sigma (x_{ij} - \mu_j)^2 > 0 \quad (1)$$

$$z_{ij} = (x_{ij} - \mu_j) / \sigma_j \quad (2)$$

Feature Engineering Taxonomy

The 111 structured metrics span six cybersecurity feature families (20 + 21 + 18 + 18 + 20 + 14 = 111 raw; 20 + 8 + 18 + 18 + 20 + 14 = 98 retained):

1. URL Lexical (20/20 retained): URL length (length_url), presence flags (email_in_url), and delimiter frequencies (qty_slash_url, qty_dot_url, qty_hyphen_url, qty_equal_url, qty_and_url). Vrbančič et al. [11] counted email_in_url among 15 custom script features; because it uses purely regex-based string matching with 0 external network requests, we classify it as lexical, leaving 14 network/resolver features.
2. Domain Structural (21/8 retained): Domain length, IP indicator, authority dots (qty_dot_domain), vowels, and hyphens (13 constant delimiter counts eliminated by zero-variance filtering).

3. Directory/Path (18/18 retained): Directory length (directory_length), slash nesting count (qty_slash_directory), and path delimiters (encoded as -1 when no directory is present).
4. File-Level (18/18 retained): Filename length (file_length) and character frequencies.
5. Query/Parameter (20/20 retained): Query length (params_length), parameter count (qty_params), and argument delimiters.
6. Resolver/Network (14/14 retained): Domain activation age (time_domain_activation), expiration time (time_domain_expiration), DNS TTL (ttl_hostname), ASN (asn_ip), lookup latency (time_response), nameservers (qty_nameservers), MX records (qty_mx_servers), resolved IPs (qty_ip_resolved), TLS certificate flag (tls_ssl_certificate), SPF records (domain_spf), redirect hops (qty_redirects), full URL Google index (url_google_index), domain Google index (domain_google_index), and URL shortener lookup flag (url_shortened). Filtering 13 constant domain delimiters leaves exactly 84 purely in-memory lexical and structural features across groups 1–5 that require strictly 0 external network lookups.

Classification Model Architectures

We evaluate five diverse classification paradigms: (i) Logistic Regression (LR) with L2 regularization (C=1.0); (ii) Decision Trees (DT) with Gini impurity splitting (max_depth=12); (iii) Random Forest (RF) ensemble with 100 bagging trees (max_depth=15); (iv) Linear SVM with Platt calibration; and (v) XGBoost [12] (100 estimators, max_depth=6, eta=0.1, subsample=0.8, colsample=0.8). Table no 2 outlines hyperparameter specifications.

Table no 2: Shows Model Hyperparameter Configurations and Optimization Settings.

Model Architecture	Hyperparameter Specification
Logistic Regression	L2 penalty, C=1.0, L-BFGS solver, max_iter=1000
Decision Tree	Gini Impurity, max_depth=12, min_samples_split=10
Random Forest	100 Trees, max_depth=15, min_samples_split=5
Linear SVM	LinearSVC (C=1.0), Platt Calibrated (3-fold CV)
XGBoost	100 Estimators, max_depth=6, η=0.1, subsample=0.8, colsample=0.8, histogram method (tree_method='hist')

Explainable AI via TreeSHAP

To explain model decisions, we deploy TreeSHAP [13], [14], which satisfies the additive attribution property f(x) = phi_0 + sum(phi_j(x)), where phi_0 = E[f(x)] is the expected base model output, and phi_j(x) is the Shapley attribution:

$$f(x) = \varphi_0 + \Sigma_{j=1}^{D'} \varphi_j(x) \quad (3)$$

$$I_j = (1 / |X_{train,exp}|) \Sigma_i |\varphi_j(x_i)| \quad (4)$$

Global feature importance I_j is computed as the mean absolute SHAP value strictly over a training explanation subset X_train,exp (N=500 random samples). Deriving I_j exclusively on D_train ensures train-only attribution for downstream feature selection. Global feature rankings demonstrate high stability across explanation subsample seeds (ρ_s = 0.9991 over all 98 features, Pearson r = 0.9995, Kendall τ = 0.9131 ± 0.026 within the Top-20, and 100% top-20 set overlap; ρ_s = 0.9995 between N=500 and N=1,000). When retraining the underlying model across 5 seeds, rank agreement remains robust (ρ_s = 0.973 ± 0.010 across 98 features, Kendall τ = 0.7948 ± 0.031 within the Top-20). Ranked features direct nested feature pruning.

Experimental Setup and Validation Protocols

All experiments are implemented in Python 3.14 utilizing scikit-learn (v1.9.1), xgboost (v3.4.1), shap (v0.52.0), and scipy (v1.18.1). Execution was conducted on an x86_64 Linux environment equipped with an Intel Core i5-3337U processor @ 1.80 GHz (2 physical cores, 4 threads) and 4 GB RAM running Ubuntu Linux. Random seed 42 is fixed across data partitioning, cross-validation folds, and subsampling for deterministic reproducibility, with multi-seed evaluations (seeds 42, 123, 456, 789, 2024; varying model tree subsampling on fixed test split) validating variance bounds. Hyperparameters are held at fixed baseline configurations (Table no 2) to evaluate architectural efficacy.

Generalization stability is evaluated using 5-Fold Stratified Cross-Validation on the training partition (N_train = 46,916), recording mean ± std across folds. Crucially, to prevent data leakage across validation folds, zero-variance

feature filtering and standard scaling transformations are fitted strictly within each training fold pipeline rather than globally prior to cross-validation. Candidate subset dimensions were evaluated by 5-fold CV on D_train, selecting k=20 as the smallest tested subset within about 0.2 pp of full performance (CV F1: 91.85% at k=5, 94.52% at k=10, 95.02% at k=20, 95.17% at k=30, and 95.20% at k=98). Both Table no 3 and the k-sweep utilize identical 5-fold stratified cross-validation splits and preprocessing pipelines; full-model CV F1 records 95.17% ± 0.13% in the multi-model benchmark (evaluated with histogram-based tree splitting, tree_method='hist') and 95.20% ± 0.17% in the standalone k-sweep (evaluated under standard exact tree construction), demonstrating cross-validation stability within 0.03 pp across splitting routines. Final model evaluation is performed on the untouched held-out test partition (N_test = 11,729). Statistical significance between classifier predictions is verified using McNemar's test with Edwards continuity correction.

IV. Results

Comparative Classifier Performance

Table no 3 summarizes the empirical performance of all five machine learning models on the held-out test partition (N_test = 11,729), alongside 5-fold cross-validation scores on the training set, PR-AUC (Average Precision), batched inference latency, and single-query latency (Fig. 3).

Table no 3: Shows Empirical Classification Performance and Latency on Held-Out Test Partition (N=11,729).

Model Architecture	Acc (%)	F1 (%)	ROC	PR (AP)	5-Fold CV F1	Batch Lat (ms/1k)	Single Lat
Logistic Regression	89.97	90.55	0.9613	0.9637	90.48 ± 0.20	1.41 ± 0.16	0.30 ms
Decision Tree	93.61	93.92	0.9668	0.9546	93.62 ± 0.17	0.53 ± 0.05	0.22 ms
Random Forest	94.30	94.57	0.9867	0.9880	94.79 ± 0.11	10.60 ± 0.31	49.38 ms
Linear SVM	89.99	90.58	0.9613	0.9637	90.45 ± 0.22	2.85 ± 0.20	4.01 ms
XGBoost	**94.82**	**95.05**	**0.9879**	**0.9889**	**95.17 ± 0.13**	**2.46 ± 0.20**	**0.71 ms**

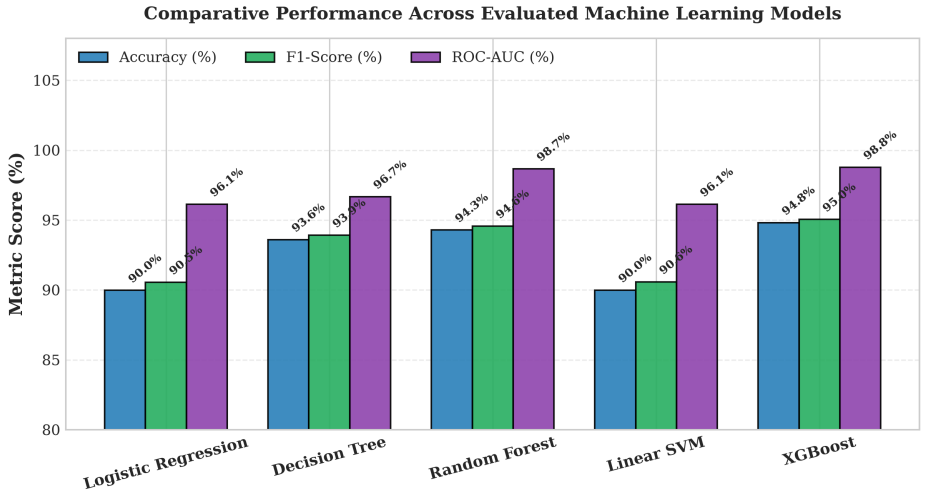


Fig. 3: Multi-Metric Classification Performance Comparison across Evaluated Architectures.

Tree-based ensemble architectures demonstrate superior discriminative capability over linear baselines. XGBoost achieves the highest overall performance: 94.82% Accuracy, 94.88% Precision, 95.22% Recall, 95.05% F1, 0.9879 ROC-AUC, and 0.9889 PR-AUC (5-fold CV F1: 95.17% ± 0.13%) with 2.46 ± 0.20 ms batch latency per 10^3 URLs and 0.71 ms single-query latency (~60× the 11.6 µs feature extraction time). Random Forest attains 94.30% Accuracy, 94.57% F1, and 0.9867 ROC-AUC (CV F1: 94.79% ± 0.11%). Decision Tree achieves 93.61% Accuracy and 93.92% F1 at 0.53 ± 0.05 ms batch latency and 0.22 ms single-query latency, with Average Precision of 0.9546. Linear models (Logistic Regression and Linear SVM) attain 89.97% and 89.99% accuracy (CV F1: 90.48% and 90.45%), producing near-identical ROC-AUC (0.9613) and PR-AUC (0.9637). This occurs because both fit regularized hyperplanes on standardized features: the cosine similarity between their weight vectors is 0.8307, and the Spearman rank correlation between their test decision scores reaches ρ_s = 0.9997 (Pearson r = 0.9836), rendering their ranking-based metrics indistinguishable at four decimal places. Discrimination diagnostics via Receiver Operating Characteristic (ROC) and

Precision-Recall (PR) curves further underscore this separation (Fig. 4).

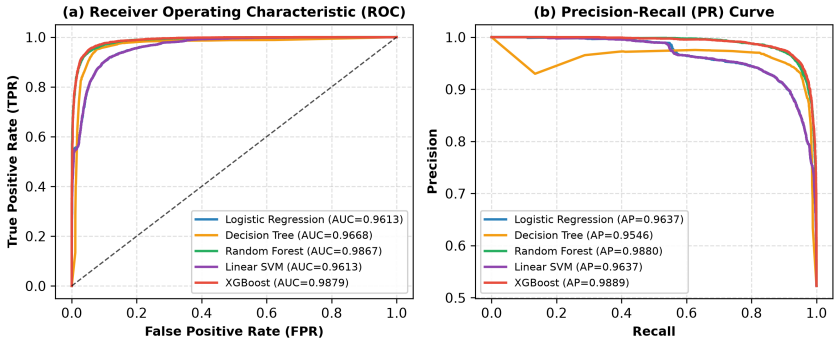


Fig. 4: Discrimination Diagnostics: (a) Receiver Operating Characteristic (ROC), and (b) Precision-Recall (PR) Curves.

Multi-Seed Robustness and Statistical Significance

On the test set, XGBoost identifies 5,836 True Positives out of 6,129 phishing URLs (Recall = 95.22%) and 5,285 True Negatives out of 5,600 benign URLs (Specificity = 94.38%), with FPR = 5.62% and FNR = 4.78%. McNemar's test with continuity correction yields chi2 = 13.79 (p = 2.04e-4 < 0.001), confirming XGBoost's statistical superiority over Random Forest. Multi-seed evaluation across 5 random seeds (seeds 42, 123, 456, 789, 2024; varying model tree subsampling on fixed test split) confirms stability: XGBoost achieves mean accuracy of 94.74% ± 0.08% and mean F1 of 94.98% ± 0.08% versus Random Forest's 94.59% ± 0.06%. Confusion matrix diagnostics on the held-out test partition reveal clear discrimination trade-offs across evaluated classifiers (Fig. 5).

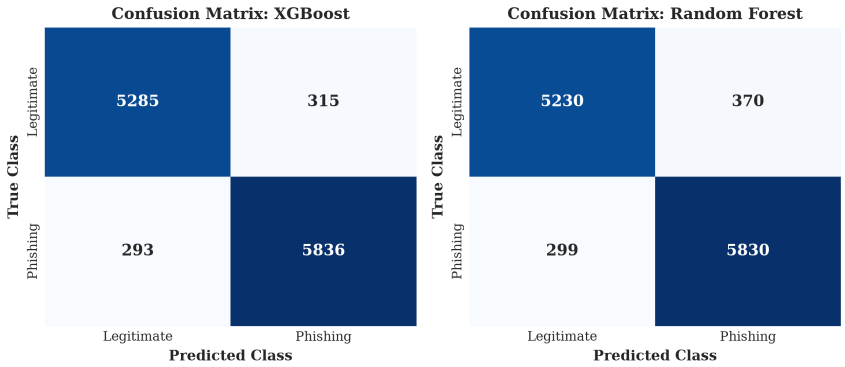


Fig. 5: Confusion Matrix Diagnostics on Test Partition across Evaluated Classifiers.

Explainability and Feature Attribution Analysis

TreeSHAP analysis was executed strictly on the training partition (N=500 random samples) to extract global feature rankings under a train-only protocol. Table no 4 details the top 10 most influential features according to mean absolute SHAP value and plausible cybersecurity semantic interpretations.

Table no 4: Shows Top 10 Influential Features Ranked by Train-Only TreeSHAP and Semantic Interpretations.

Rank	Feature Identifier	Mod.	|SHAP|	Plausible Cybersecurity Interpretation
1	directory_length	Lex	1.0811	Path length/presence (deep nesting masks fraud; -1 = no path)
2	time_domain_activation	Net	0.9912	Domain age (new domains/failed WHOIS signal throwaways)
3	qty_slash_url	Lex	0.7377	Forward slash delimiter count across URL hierarchy
4	length_url	Lex	0.7222	Raw URL character length (used to obscure landing)
5	qty_dot_domain	Lex	0.5293	Subdomain dot count (used in multi-level brand spoofing)
6	ttl_hostname	Net	0.3147	DNS TTL metric (short TTLs reflect fast-flux evasive DNS)
7	asn_ip	Net	0.2057	Autonomous System Number (flags bulletproof/rogue ASNs)
8	time_response	Net	0.1724	HTTP lookup response latency in seconds (host variability)
9	qty_hyphen_directory	Lex	0.1277	Hyphens in path (inserted to mimic trusted brand keywords)
10	qty_redirects	Net	0.1082	HTTP redirect hop count (multi-hop chains obscure target)

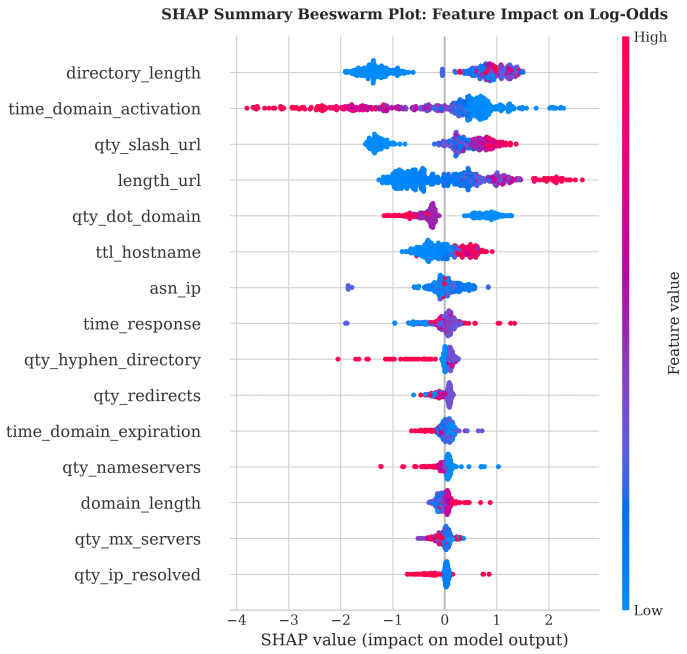


Fig. 6: TreeSHAP Global Summary Beeswarm Plot Derived Exclusively from Training Instances.

Fig. 6 illustrates directional attributions across training instances. Crucially, the interpretation of directory_length and time_domain_activation must be contextualized with -1 sentinel encoding. Because absence of directory (affecting 60.29% of legitimate test URLs and 59.86% in the training set) or failed WHOIS resolution is coded as -1, tree decision boundaries split on whether a feature is positive versus <= 0, effectively encoding qualitative presence alongside quantitative magnitude within a single scalar dimension. Beyond global rankings, the framework generates granular, sample-level waterfall explanations for individual triage decisions (Fig. 7).

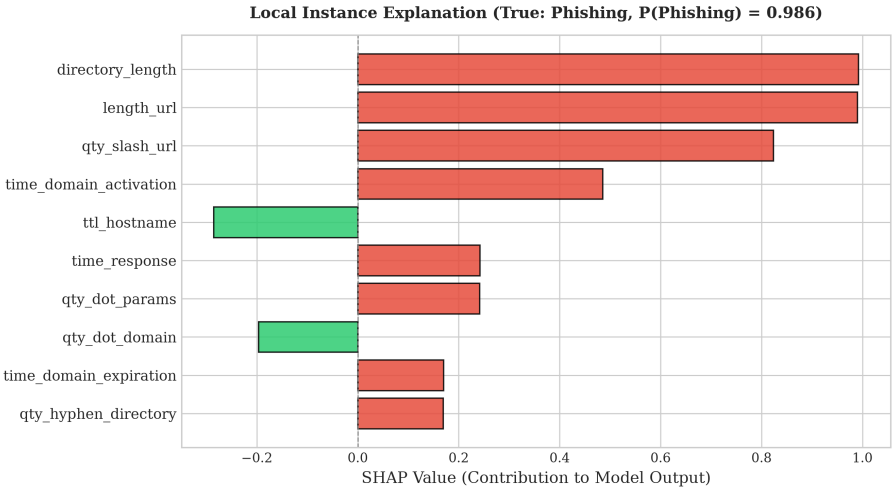


Fig. 7: Local Prediction Explanation Waterfall Plot for Representative Phishing Instance.

Feature Reduction and Computational Benchmarking

To assess whether compact feature subsets maintain classification performance, we evaluated XGBoost across nested feature subsets ranked by training set SHAP importance: Full (98 features), Top-30, Top-20, Top-10, Top-5, alongside a dedicated Lexical-Only baseline (84 features, 0 network lookups). Table no 5 delineates performance, latency, memory footprint, and throughput trade-offs across evaluated feature subsets (Fig. 8).

Table no 5: Shows Feature Reduction Trade-Offs, Model Footprint, and Computational Throughput Profiles (Seed 42).

Configuration	Net/Lex	Acc (%)	F1 (%)	ROC	Lat (ms/1k)	Size (MB)	Throughput (q/s)
XGBoost (Full 98)	14N/84L	94.82	95.05	0.9879	2.46 ± 0.20	0.126	406,318
XGBoost (Top-30)	12N/18L	94.74	94.97	0.9882	1.83 ± 0.07	0.134	546,448
XGBoost (Top-20)	**11N/9L**	**94.82**	**95.05**	**0.9876**	**1.67 ± 0.06**	**0.130**	**600,099**
XGBoost (Top-10)	5N/5L	94.19	94.48	0.9859	1.57 ± 0.11	0.126	636,942
XGBoost (Top-5)	1N/4L	91.32	91.84	0.9723	1.59 ± 0.12	0.121	628,930
XGBoost (Lexical-84)	**0N/84L**	**89.68**	**90.04**	**0.9627**	**3.16 ± 0.43**	**0.342**	**316,002**
Random Forest	14N/84L	94.30	94.57	0.9867	10.60 ± 0.31	4.068	94,369
Decision Tree	14N/84L	93.61	93.92	0.9668	0.53 ± 0.05	0.029	1,884,868
Linear SVM	14N/84L	89.99	90.58	0.9613	2.85 ± 0.20	0.006	350,920
Logistic Regression	14N/84L	89.97	90.55	0.9613	1.41 ± 0.16	0.004	708,898

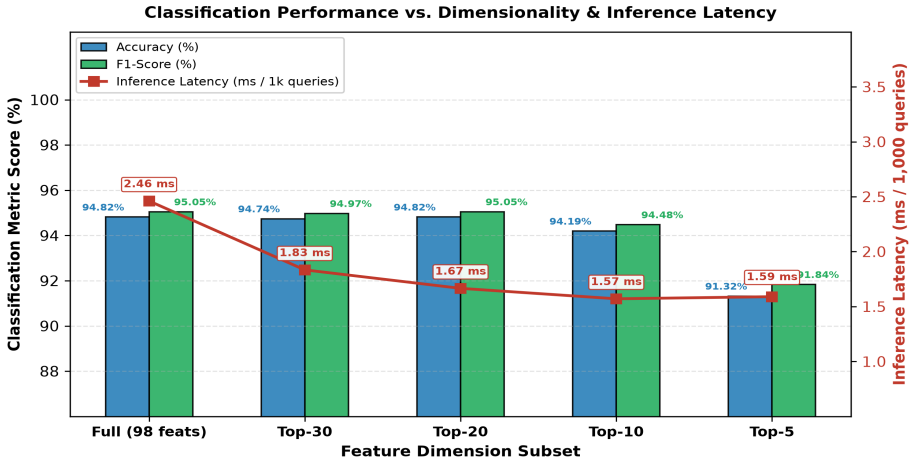


Fig. 8: Classification Accuracy and Inference Latency vs. Feature Dimensionality (k-Sweep).

Two-Tier Uncertainty-Gated Cascade Evaluation

To empirically validate a practical two-tier architecture, we evaluated an uncertainty-gated cascade: when the Tier 1 lexical model's predicted probability falls within an uncertainty band $[\tau_L, \tau_H]$, classification is deferred to the Tier 2 Top-20 resolver model. Operating thresholds were calibrated via 5-fold cross-validation on the training set to maximize query offloading while constraining the Tier-1 out-of-fold phishing leakage rate below 2.5% (CV sweep at [0.10, 0.90]: 64.21% coverage, 97.98% accuracy, 2.35% phishing leak rate). On the held-out test partition (N=11,729), Tier 1 autonomously resolves 64.28% of instances (7,539 URLs) purely in-memory with 97.78% accuracy, eliminating external lookups for nearly two-thirds of incoming traffic. Specifically, Tier 1 classifies 3,380 URLs as legitimate ($\hat{p} < 0.10$) with 90 false negatives, yielding a test phishing leakage rate of 2.66% (slightly above the 2.5% CV design target due to test distribution variance), and 4,159 URLs as phishing ($\hat{p} > 0.90$) with 4,082 true phishing. This yields a conditional Tier-1 phishing recall of 97.84% (4,082/4,172) on the resolved subset; however, accounting for the 1,957 deferred phishing URLs and 90 Tier-1 leaks, the end-to-end cascade achieves an overall FNR of 4.98% (305/6,129) and FPR of 5.93% (332/5,600), compared to 4.78% FNR and 5.62% FPR for monolithic full-network XGBoost. Tier 1 resolves 3,051 pathless legitimate, 316 legitimate with paths, 89 pathless phishing, and 4,083 phishing with paths. Notably, of the 141 pathless phishing URLs in the test set, 88 (62.4%) are predicted legitimate at Tier 1 ($\hat{p} < 0.10$) and bypass Tier 2 entirely, highlighting an operational blind spot for root-domain phishing. The deferred 4,190 ambiguous URLs are evaluated by Tier 2, which achieves 88.78% accuracy on these difficult instances. Overall pipeline accuracy reaches 94.57% and F1-score reaches 94.81%—matching within 0.25 pp of monolithic full-network accuracy while saving 64.28% of resolver lookups. Table no 6 presents the operational breakdown of the cascade system.

Table no 6: Shows Two-Tier Uncertainty-Gated Cascade Performance, Resolver Offloading, and Baseline Comparison.

Architecture / Stage	Target Traffic Subset	Coverage (%)	Accuracy (%)	Phishing F1 (%)	External Lookups Saved
Monolithic XGBoost (Full 98)	Full Test Traffic (11,729 URLs)	100.0%	94.82%	95.05%	0.0% (Baseline)
Cascade Tier 1 (Lexical-84)	Decisive Cases ($\hat{p} < 0.10$ or $\hat{p} > 0.90$)	**64.28% (7,539)**	**97.78%**	**98.00%**	**64.28% Eliminated**
Cascade Tier 2 (Top-20 Net)	Deferred Borderline ($0.10 \le \hat{p} \le 0.90$)	35.72% (4,190)	88.78%	88.11%	Executed as Needed
Cascade Hybrid Pipeline	End-to-End Operational Traffic	**100.0%**	**94.57%**	**94.81%**	**64.28% Total Savings**
Directory Heuristic Baseline	directory_length > -1 Rule	100.0%	79.84%	83.51%	N/A (Heuristic)

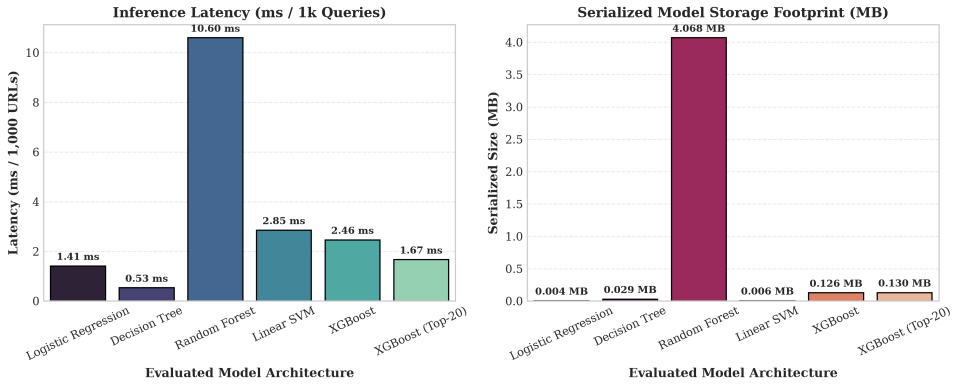


Fig. 9: Computational Resource Profiling: Batched Inference Latency and Serialized Model Disk Footprint across Evaluated Architectures.

Path-Presence Heuristic and Lexical Ablation

As a baseline, a heuristic classifying URLs as phishing whenever a directory path exists (directory_length > -1) achieves 79.84% accuracy, 83.51% F1-score, and 97.70% recall, exploiting dataset sampling skew (97.70% of phishing vs 39.71% of legitimate URLs contain paths). On the 8,212 path-bearing test URLs, where the naive heuristic achieves only 72.92% accuracy, the full XGBoost model attains 93.59% accuracy (Recall = 96.34%, F1 = 95.64%), demonstrating genuine discriminative learning beyond path presence. On the 3,517 pathless URLs, XGBoost achieves 97.67% accuracy and 99.76% specificity (Recall = 47.52%, F1 = 62.04%). An ablation removing all 56 path, directory, file, and parameter features ('no path/file/query-group features') yielded a 28-feature URL/domain lexical model that achieves 89.17% accuracy (86.23% on path-bearing, 96.05% on pathless) and 89.58% F1-score. When deployed in Tier 1 of the cascade at [0.10, 0.90], this model resolves 62.48% of test queries (7,328 URLs) with 97.69% accuracy, sustaining 94.53% overall cascade accuracy and 94.78% F1-score, confirming that non-path lexical cues reduce reliance on explicit path tokens while still supporting a tiered design. Computational resource profiling across memory footprint and execution latency confirms operational feasibility for deployment (Fig. 9).

V. Discussion

Operational Considerations and In-Memory Triage

A critical operational factor in ML phishing detection is distinguishing feature extraction latency from model inference latency. While model inference takes 1.67 ms per 1,000 queries (model-only), external resolver queries introduce substantial round-trip network delays—typically tens to hundreds of milliseconds for synchronous DNS resolution, WHOIS TCP queries, and TLS handshakes (empirically measured at 93.8 ms median, 150.5 ms mean across 50 distinct global domains; see also [5], [6])—as well as strict query rate limits, making synchronous external queries impractical for high-volume inline network inspection. In contrast, lexical URL features are parsed in-memory at 11.6 µs per URL with zero external overhead. This motivates and empirically validates the evaluated two-tier perimeter defense: Tier 1 resolves 64.28% of instances with 97.78% accuracy in-memory, while Tier 2 executes only for the remaining borderline cases, sustaining 94.57% pipeline accuracy while cutting resolver load by nearly two-thirds.

Quantitative Error Analysis

Analysis of 293 False Negatives (FN = 293, 4.78% of phishing test samples) and 315 False Positives (FP = 315, 5.62% of legitimate test samples) reveals distinct structural patterns influenced by -1 sentinel distributions:

1. False Negatives: Phishing URLs misclassified as legitimate exhibited shallow or absent directory paths (median 1.0 char, mean 6.57 vs. median 24.0, mean 29.57 in TP), shorter overall length (median 22.0 vs. 48.0 in TP), and fewer slashes (median 1.0 vs. 3.0 in TP). Crucially, 44.71% of FNs (131/293) had failed WHOIS lookups resulting in -1 sentinels for domain activation age, compared to 35.32% in TPs (2,061/5,836), a statistically significant difference ($\chi^2 = 10.31$, $p = 0.0013$). Including -1 sentinels, median domain age was 159.0 days for FNs vs 682.5 days for TPs. Excluding -1 sentinels, valid FN domains averaged 3,351.66 days (median 2,702.50 days, N=162) versus valid TP domains averaging 2,313.14 days (median 1,914.00 days, N=3,775). FNs evade detection through two mechanisms: lookup failures that collapse into default -1 sentinels, and hosting on aged, compromised legitimate infrastructure.
2. False Positives: Legitimate URLs misclassified as phishing featured elongated directory paths (median 13.0 chars, mean 18.55 vs. median -1.0, mean 3.41 in TN; 97.5% of FPs had paths vs. only 36.3% of TNs), longer URLs (median 33.0 vs. 18.0 in TN), and directory hyphens (mean 0.63 vs. -0.49 in TN; excluding -1 sentinels, FP mean was 0.67 vs. 0.41 in TN), typical of single-page apps, deep tracking links, and UTM parameter structures.

Limitations and Threats to Validity

Key limitations and threats to validity include: (1) Sampling Artifacts & Row Duplication: legitimate URLs originate from Alexa top domains and search indices (where 60.29% assign -1 for directory absence), while phishing URLs originate from PhishTank (97.70% contain sub-paths). An exact cross-partition audit identified 335 instances (2.86% of test) sharing identical feature vectors with training rows, 99.1% of which share identical labels. Evaluating on the strictly de-duplicated test subset (N=11,394) preserves 94.71% accuracy and 95.05% F1-score for XGBoost (94.18% / 94.58% for RF), confirming that row repetition does not alter conclusions; (2) Adversarial Evasion and Root-Domain Blind Spots: because Tier 1 relies heavily on lexical and path metrics, adversaries hosting phishing pages directly at root domains can evade Tier 1 triage—88 of 141 pathless phishing test URLs (62.4%) were misclassified as legitimate at Tier 1 and bypassed Tier 2. Attackers aware of path-heavy features can also craft minimal-path URLs or manipulate delimiter frequencies to manipulate SHAP attributions; (3) Temporal Age & Concept Drift: the dataset dates from 2020, requiring ongoing validation against modern evasion tactics (e.g. serverless proxies, IPFS gateways); (4) Lookup Reliability: external WHOIS/DNS queries may experience rate-limiting, collapsing metrics into -1 sentinels; (5) Hyperparameter Tuning: models were evaluated on fixed baseline configurations; and (6) Base-Rate Fallacy & Operational Prevalence: while the benchmark reflects balanced data (~52.2% phishing), in production traffic with ≤ 1% phishing prevalence, XGBoost's 95.2% recall and 5.6% FPR yield ~14.6% empirical precision (~55,600 false alarms per million URLs), necessitating conservative operational thresholds, tiered cascaded filtering, or secondary telemetry.

VI. Conclusion

In this paper, we presented an empirical benchmark and explainable machine learning framework for phishing website detection using 111 structured URL lexical, directory path, file, query, and resolver features. Evaluated on 58,645 URLs, XGBoost achieved superior predictive performance: a 5-seed mean of 94.74% ± 0.08% accuracy and 94.98% ± 0.08% F1-score (seed 42: 94.82% Accuracy, 94.88% Precision, 95.22% Recall, 95.05% F1-score, and 0.9879 ROC-AUC on 11,729 held-out test samples; 5-fold CV F1: 95.17% ± 0.13%), demonstrating statistically significant superiority over Random Forest via McNemar's test ($p < 0.001$).

By integrating TreeSHAP strictly on training data to ensure train-only attribution, the framework provides transparent and stable ($\rho_s = 0.9991$ Spearman rank correlation, Kendall $\tau = 0.9131$ within Top-20; 100% top-20 set overlap) feature attributions, identifying directory length, domain age, delimiter frequency, and URL length as primary discriminative signals. Systematic feature reduction, guided by training cross-validation (elbow at k=20, selected as the smallest tested subset within 0.2 pp of full performance), revealed that pruning 79.6% of features (from 98 to 20) preserves full predictive performance (5-seed test mean: 94.71% ± 0.09% accuracy, 94.95% ± 0.08% F1; seed 42: 94.82% accuracy, 95.05% F1) while lowering model-only inference latency to 1.67 ± 0.06 ms per 1,000 queries. Furthermore, evaluating an uncertainty-gated cascade demonstrates that a zero-network lexical-only model (89.68%

accuracy, 90.04% F1) resolves 64.3% of queries purely in-memory with 97.78% accuracy, preserving 94.57% overall pipeline accuracy while cutting external network queries by 64.3%, establishing an empirically validated tiered defense for high-throughput perimeter triage.

Future work will explore online adaptive learning for concept drift, character-level embeddings for IDN homograph detection, and open-source browser and DNS gateway extensions.

Data and Code Availability

To ensure complete scientific reproducibility and open evaluation, all source code, preprocessing routines, model training scripts, feature ranking pipelines, and serialized benchmark metrics are available in the public project repository at: https://github.com/tanaychess/PhishingURLDetector.

Acknowledgment

The author acknowledges the Department of Computer Science, Bhonsala Military College, for providing research infrastructure and computational resources for this investigation.

References

[1]. Anti-Phishing Working Group (APWG), 'Phishing Activity Trends Report, 4th Quarter 2023,' APWG, Tech. Rep., 2023.

[2]. Verizon Business, '2024 Data Breach Investigations Report (DBIR),' Verizon, Tech. Rep., 2024.

[3]. D. Sahoo, C. Liu, and S. C. H. Hoi, 'Malicious URL detection using machine learning: A survey,' arXiv:1701.07179, 2017.

[4]. R. S. Rao and A. R. Pais, 'Detection of phishing websites using an efficient feature-based machine learning framework,' Neural Comput. & Appl., vol. 31, no. 8, pp. 3851-3873, 2019.

[5]. A. Hannousse and S. Yahiouche, 'Towards benchmark datasets for machine learning based website phishing detection: An experimental study,' Eng. Appl. Artif. Intell., vol. 104, p. 104347, 2021.

[6]. S. Marchal et al., 'Off-the-hook: An efficient and usable client-side phishing prevention application,' IEEE Trans. Comput., vol. 66, no. 10, pp. 1717-1733, 2017.

[7]. O. K. Sahingoz, E. Buber, O. Demir, and B. Diri, 'Machine learning based phishing detection from URLs,' Expert Syst. Appl., vol. 117, pp. 345-357, 2019.

[8]. M. Babagoli, M. P. Aghababa, and V. Solouk, 'Heuristic nonlinear regression strategy for detecting phishing websites,' Soft Comput., vol. 23, pp. 4315-4327, 2019.

[9]. C. Opara, B. Wei, and Y. Chen, 'HTMLPhish: Enabling phishing web page detection by applying deep learning techniques on HTML analysis,' in Proc. IJCNN, 2020, pp. 1-8.

[10]. M. A. Adebowale, K. T. Lwin, and M. A. Hossain, 'Intelligent phishing detection scheme using deep learning algorithms,' J. Enterp. Inf. Manag., vol. 36, no. 3, pp. 747-766, 2023.

[11]. G. Vrbančič, I. Fister Jr, and V. Podgorelec, 'Datasets for phishing websites detection,' Data in Brief, vol. 33, p. 106438, 2020.

[12]. T. Chen and C. Guestrin, 'XGBoost: A scalable tree boosting system,' in Proc. ACM SIGKDD, 2016, pp. 785-794.

[13]. S. M. Lundberg et al., 'From local explanations to global understanding with explainable AI for trees,' Nature Mach. Intell., vol. 2, no. 1, pp. 56-67, 2020.

[14]. S. M. Lundberg and S.-I. Lee, 'A unified approach to interpreting model predictions,' in NeurIPS, vol. 30, pp. 4765-4774, 2017.

[15]. R. M. Mohammad, F. Thabtah, and L. McCluskey, 'Predicting phishing websites based on self-structuring neural network,' Neural Comput. & Appl., vol. 25, no. 2, pp. 443-458, 2014.

[16]. A. Al-Alyan and S. Al-Ahmadi, 'Robust URL phishing detection based on deep learning,' KSII TIIS, vol. 14, no. 7, pp. 2752-2768, 2020.

[17]. K. L. Chiew et al., 'A new hybrid ensemble feature selection framework for machine learning-based phishing detection system,' Inf. Sci., vol. 484, pp. 153-166, 2019.

[18]. M. Al-Fayoumi, B. Alhijawi, Q. Abu Al-Haija, and R. Armoush, 'XAI-PhD: Fortifying trust of phishing URL detection empowered by Shapley additive explanations,' iJOE, vol. 20, no. 11, pp. 80-101, 2024.

[19]. K. M. M. Uddin et al., 'Explainable machine learning for phishing site detection: A high-efficiency approach using boosting models and SHAP,' J. Eng., vol. 2025, no. 1, 2025.

[20]. M. C. Calzarossa, P. Giudici, and R. Zieni, 'Explainable machine learning for phishing feature detection,' Qual. Reliab. Eng. Int., vol. 40, no. 1, pp. 362-373, 2024.

[21]. T. Kehkashan et al., 'Explainable phishing website detection for secure and sustainable cyber infrastructure,' Sci. Rep., vol. 15, p. 41751, 2025.